

GREENFIELD INSTITUTE — COURSE REGISTRATION SYSTEM

Group 11- Members

- ✓ Kimathi Nkatha Winfred- SCM211-0370/2024
- ✓ Erick Mwendwa Ndeto- SCM211-0367 /2024
- ✓ Lewis Gitonga- SCM211-0363/2024
- ✓ Allan Kimani- SCM211-0316/2024
- ✓ Mary Mwangi-SCM211-0299/2024

PROJECT REPORT

1. Problem Statement

Greenfield Institute previously processed course registrations through emails and manually maintained spreadsheets. The result was duplicate registrations, stale course-availability information, and a poor experience for both students and administrators. This project replaces that workflow with a centralised, web-based registration system built on a clean three-tier architecture.

2. Three-Tier Architecture

The application is organised as three independent layers, each with a single responsibility. Each tier only communicates with the tier directly below it. The browser never speaks to MySQL; PHP never renders HTML for the dynamic pages — it returns JSON (or XML for the data-exchange endpoint), and the browser builds the user interface from that data.

2.1 Presentation Tier — HTML, CSS, and JavaScript

The presentation tier runs entirely in the user's browser. It is built using vanilla HTML, CSS, and JavaScript, with no frontend frameworks. All dynamic interactions use the modern fetch API to call the PHP business-logic tier over AJAX, returning JSON data which the JavaScript then renders into the page.

The system contains the following pages:

The login and sign-up page (index.html) is a combined view with a 3.5-second diagonal-split animation on desktop. New students sign up by entering the school email and registration number their administrator pre-registered them with, plus a chosen password.

Students have access to a personalised dashboard which shows a profile-completion banner, two stat cards (enrolled courses and profile completion percentage), a scrollable table of their

current registered courses with column headings, and a grid of recommended courses based on their department.

The course catalogue page offers live keyword search and a department filter using a custom themed dropdown. The custom dropdown was built because native HTML select elements only allow the closed trigger to be styled — the option panel that drops down is rendered by the operating system and cannot be themed to match the brand. The custom dropdown is ARIA-compliant (keyboard navigation, click-outside, focus management).

The "my courses" page displays the student's registered units in a horizontally-scrollable table with status pills and a Drop action.

The profile and settings pages let students edit their optional profile fields and change their password (with current-password verification).

The administrator panel contains an overview dashboard (action items strip showing pending registrations and students still to sign up, summary statistics, an activity feed of recent events, and capacity-alert cards for courses that are full or filling up), a students management page for pre-registering admitted students, a course catalogue management page with full create/read/update/delete plus XML import via file upload and dated XML export, and a registrations page where pending requests can be approved or rejected, and already-processed registrations can be dropped to fix mistakes.

Shared client-side assets include a single responsive stylesheet using a mint Greenfield palette (#2DD49B primary, off-white background), a thin AJAX helper that wraps `fetch()` and sets the X-Requested-With header so PHP can detect AJAX calls, a navigation module that toggles the off-canvas sidebar drawer on phones, a custom dropdown component, and an authentication module that drives the login-to-signup transition animation.

Responsive design was a major focus of the project. The sidebar navigation collapses into a hamburger-triggered slide-out drawer at 1199 pixels and below, with a dimming backdrop that closes the sidebar on click. The auth screens swap from a diagonal-split layout on desktop to a stacked image-on-top, form-below layout on phones. Wide data tables (current courses, my courses, admin students, admin course catalogue, admin registrations) are wrapped in a scroll container that handles horizontal overflow on phones while preserving column headers. Toolbar controls (search input and filter dropdown) stack to full-width controls at 480 pixels and below.

2.2 Business-Logic Tier — PHP

The PHP tier is the only place where business rules live. Validation happens server-side because clients can be tampered with. All database access goes through PDO with prepared statements, defeating SQL injection. Cross-site scripting is mitigated by escaping every dynamic value before

it reaches the DOM (`escapeHtml` in the client-side JavaScript helper, `htmlspecialchars` in PHP). Passwords are stored as bcrypt hashes via `password_hash()` and `password_verify()`. Login returns a generic "invalid credentials" message regardless of which field was wrong, preventing user enumeration. Session IDs are regenerated on login to prevent session fixation.

Shared PHP modules include a database connector that returns a configured PDO instance, an authentication module providing `require_login()` and `require_admin()` helpers (every protected endpoint calls one of these at the top), a JSON response helper, and a request-body parser.

The API endpoints accept JSON in their request body and return JSON. The endpoints are: `register.php` (completes sign-up for a pre-registered student), `login.php` (authenticates email and password, regenerates session ID), `logout.php` and `me.php` (session teardown and current-user lookup), `dashboard.php` (single round-trip that returns stats, current courses, recommendations, and profile-completion percentage for the student dashboard), `courses.php` (lists, searches, and filters courses with live enrollment count and is-full flag), `enroll.php` (registers a student with capacity enforcement and transactional row locking), `drop.php` and `my_courses.php` for student self-service, `profile.php` and `change_password.php` for profile edits and password changes, `admin_overview.php` (powers the admin landing page with action items, totals, capacity alerts, and activity feed), `admin_students.php` (pre-register, list, and delete student stubs), `admin_courses.php` for admin CRUD on courses, `admin_registrations.php` (approve, reject, and drop registrations — drop works on already-approved or already-rejected rows to fix mistakes), `courses_xml.php` for XML export, and `import_xml.php` for admin-only XML import.

The system enforces role-based access control. Every page that needs authentication calls `require_login()` or `require_admin()` at the top of the file. Administrators landing on the student dashboard are redirected to the admin dashboard, and vice-versa.

2.3 Data Tier — MySQL

The database contains three tables, defined in `sql/greenfield.sql`.

The `users` table holds both students and administrators in a single table, distinguished by a `role` column. The `password_hash` column is nullable so that administrators can create student stubs (name, email, registration number) without a password, which the student then fills in at sign-up. Five optional profile fields (`registration_number`, `year_of_birth`, `gender`, `department`, `programme`) are added or edited over time.

The `courses` table holds the course catalogue with `code`, `title`, `description`, `instructor`, `capacity`, and `department` columns.

The registrations table is a many-to-many link between users and courses. The status column tracks the registration through its lifecycle: pending (set when a student first registers and awaits admin review), approved (admin has confirmed enrolment), rejected (admin denied the request), and dropped (student or admin pulled the registration after approval). A composite unique constraint on (user_id, course_id) prevents a student from having two rows for the same course at the database level. Foreign keys with ON DELETE CASCADE keep the data consistent if a user or course is removed. Re-registering after a drop reuses the same row by flipping the status back to pending.

Incremental schema changes are captured as named migration scripts in the sql folder (add_profile_fields.sql, registration_status.sql, drop_credits.sql, student_preregistration.sql), so an existing deployment can be brought forward without re-importing the whole database.

3. Use of XML

The case study calls for XML to demonstrate structured-data exchange. The system supports XML in both directions.

Export is performed by a GET request to /api/courses_xml.php, which builds a well-formed XML document from the live database using PHP's DOMDocument and serves it with an application/xml MIME type. The administrator's manage_courses.php page links to it as a one-click "Export XML" download. The file is also consumable by partner systems or backup scripts without changing the business-logic tier.

Import is performed by a POST request to /api/import_xml.php (administrator-only), which accepts either an uploaded file or the bundled data/courses_sample.xml. It parses the document with simplexml_load_file, then merges each course node into the courses table with an upsert (INSERT ... ON DUPLICATE KEY UPDATE). A bulk-load workflow that previously meant copy-pasting from a spreadsheet is now one click.

The XML structure is intentionally human-readable so a non-developer can edit it in any text editor.

4. Request Flow Example — Registering for a Course

When a student clicks the Register button in the course catalogue, JavaScript calls POST /api/enroll.php with the course_id as JSON through the api.post() helper. The enroll endpoint checks the session via require_login(), opens a database transaction, locks the course row with SELECT ... FOR UPDATE, and verifies three rules: the course exists, the student is not already pending or approved for it, and the current enrollment (counting pending plus approved seats) is less than the capacity.

If all three rules pass, the endpoint inserts a row into the registrations table with status "pending", commits the transaction, and returns a success message. The browser then shows a flash banner and re-fetches the catalogue, which now reports the updated enrollment count and available seats.

Later, an administrator visits the admin registrations page and clicks Approve. POST `/api/admin_registrations.php` flips the row's status to "approved" — but only if it is still "pending", guarding against double-approval. The administrator can subsequently click Drop to undo the approved registration if the student picked the wrong course by mistake.

If any rule fails (course full, already registered, course missing), PHP returns the appropriate HTTP status and a human-readable error, which the browser surfaces in a flash banner. No business decision is made in the browser.

5. Additional Features Implemented

Pre-registration and sign-up: administrators add admitted students (name, email, registration number) ahead of time, and the student then signs up by entering the same email and registration number plus a chosen password on the combined login and sign-up page.

Approval workflow: every student registration enters as "pending" and waits for an administrator decision (approved or rejected). Already-processed registrations can later be dropped by either the student or an administrator.

Search and filter: keyword and department filtering on the catalogue, with a 200-millisecond debounce so the server is hit at most once per typing burst.

AJAX everywhere: no full page reloads after the initial HTML. Every interaction is a fetch call returning JSON. Verifiable by opening the browser's developer tools network tab and watching the requests fire while using the site.

Responsive design: tested down to approximately 320 pixel viewport width. Includes an off-canvas sidebar, hamburger toggle, stacked auth layout, horizontally-scrollable data tables with column headings, and adaptive toolbars.

Profile completion tracking: five optional profile fields contribute to a percentage that drives a dashboard nag banner and a recommendation engine. The recommender prefers courses in the student's department, falling back to any available course.

Recommended-for-you list: the dashboard shows up to three courses the student has not yet registered for, prioritised by their department if known.

Enhanced administrator dashboard: action-items strip (pending registrations, students still to sign up), summary totals (students, courses, active registrations), an activity feed of recent events, and capacity-alert cards for courses that are full or filling up.

Secure password handling: bcrypt via password_hash and password_verify, generic "invalid credentials" error to prevent user enumeration, session ID regeneration on login, and current-password verification on password change.

Role-based access control: students cannot reach administrator pages; administrators are routed to the administrator dashboard on login.

6. Setup Instructions

The project runs on a standard XAMPP install (Windows, macOS, or Linux).

Begin by installing XAMPP and starting the MySQL module from the XAMPP control panel. Apache is optional — you can use PHP's built-in web server instead.

Create the database by opening phpMyAdmin (at <http://localhost/phpmyadmin>), creating a database called greenfield_db, clicking into it, then on the Import tab choosing sql/greenfield.sql and clicking Go. The schema and seed data load together. The seeded password for every demo account is "password123" — bcrypt hashes are baked into the SQL file.

Configure credentials by copying includes/config.example.php to includes/config.php and editing DB_HOST, DB_USER, DB_PASS, DB_NAME if your MySQL setup differs from the XAMPP defaults (localhost, root, no password, greenfield_db).

Serve the site by running, from the project root: "C:\xampp\php\php.exe" -S localhost:8000 (or any PHP-capable web server — Apache via htdocs works too).

Open <http://localhost:8000/> and log in.

A production deployment is also live on InfinityFree shared hosting at <https://greenfield-institute.infinityfreeapp.com/>.

CAUTION!! Test this link in Edge browser **ONLY**. *The project is hosted on InfinityFree free hosting. Google Chrome and Brave browser may show a Safe Browsing warning because the shared hosting IP has been associated with phishing on unrelated free-tier sites. The site loads cleanly in Microsoft Edge.*

The test credentials are:

- administrator account: admin@greenfield.edu password : "password123"
- test student account at test@student.greenfield.edu password : "testing123".

The same SQL file is imported into the hosted MySQL database, and includes/config.php is updated with the InfinityFree credentials.

7. Submission Contents

The submission folder includes:

- the index.html and signup.html files for the combined login and sign-up screen
- the student pages (dashboard.html, courses.html, my-courses.html, profile.html, settings.html)
- the administrator pages under the admin folder (dashboard.php, students.php, manage_courses.php, registrations.php)
- the responsive stylesheet at assets/css/style.css
- the shared AJAX client at assets/js/api.js
- the UI behaviour modules at assets/js/nav.js, dropdown.js, and auth.js
- the PHP business-logic endpoints under the api folder
- the database, configuration, and authentication shared modules under the includes folder
- a sample XML file at data/courses_sample.xml used by the import endpoint;
- the database schema and seed data at sql/greenfield.sql, with additional migration scripts in the same folder
- and this report.

8. Conclusion

The system replaces Greenfield Institute's manual, spreadsheet-driven registration process with a maintainable web application. Splitting the code along the three-tier boundary means each layer can evolve on its own: the frontend could be re-skinned (or replaced with a mobile app) without touching PHP; the business logic could be ported to another language without touching the database; and the database can be migrated or scaled without affecting any client. The XML import and export adds a cleanly defined data-exchange surface that lets Greenfield interoperate with other systems without exposing its internal database schema. The approval and pre-registration workflows close the loop on the original problems (duplicate registrations, stale availability, inaccurate records), and the responsive design pass means the same system works on a phone in a corridor or on a desktop in the registrar's office.
